

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 200)

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)

print("Loaded")
```

Loaded

```
In [2]: df = pd.read_csv('games.csv')
df.head()
```

Out [2]:

	AppID	Name	Release date	Estimated owners	Peak CCU	Required age	Price	DiscountDLC count
2539430	Black Dragon Mage Playtest	Aug 1, 2023	0 - 0	0	0	0.00	0	0
496350	Supipara - Chapter 1 Spring Has Come!	Jul 29, 2016	0 - 20000	0	0	5.24	65	0
1034400	Mystery Solitaire The Black Raven	May 6, 2019	0 - 20000	0	0	4.99	0	0
3292190	버튜버 파 라노이아 - Vtuber Paranoia	Oct 31, 2024	0 - 20000	1	0	8.99	0	1
3631080	Maze Quest VR	Apr 24, 2025	0 - 20000	0	0	4.99	0	0

In [3]: `df.keys()`

Out [3]: Index(['AppID', 'Name', 'Release date', 'Estimated owners', 'Peak CCU', 'Required age', 'Price', 'DiscountDLC count', 'About the game', 'Supported languages', 'Full audio languages', 'Reviews', 'Header image', 'Website', 'Support url', 'Support email', 'Windows', 'Mac', 'Linux', 'Metacritic score', 'Metacritic url', 'User score', 'Positive', 'Negative', 'Score rank', 'Achievements', 'Recommendations', 'Notes', 'Average playtime forever', 'Average playtime two weeks', 'Median playtime forever', 'Median playtime two weeks', 'Developers', 'Publishers', 'Categories', 'Genres', 'Tags', 'Screenshots', 'Movies'], dtype='object')

The CSV files has broken header. The header row has 39 names, but every data row has 40 values, seems like because the csv merged Discount and DLC count into one header name DiscountDLC count

```
In [4]: # Splitting DLC count and Discount
df.columns = ['Name', 'Release date', 'Estimated owners', 'Peak CCU',
              'Required age', 'Price', 'Discount', 'DLC count', 'About the game',
              'Supported languages', 'Full audio languages', 'Reviews',
              'Header image', 'Website', 'Support url', 'Support email', 'Windows',
              'Mac', 'Linux', 'Metacritic score', 'Metacritic url', 'User score',
              'Positive', 'Negative', 'Score rank', 'Achievements', 'Recommendation',
              'Notes', 'Average playtime forever', 'Average playtime two weeks',
              'Median playtime forever', 'Median playtime two weeks', 'Developers',
              'Publishers', 'Categories', 'Genres', 'Tags', 'Screenshots', 'Movies']

print("Done")

# got the error of length mismatch. so instead of adding one more col, I just
```

Done

```
In [5]: pd.set_option('display.max_columns', None)
df.head()
```

Out [5]:

	Name	Release date	Estimated owners	Peak CCU	Required age	Price	Discount	DLC count	Abc
2539430	Black Dragon Mage Playtest	Aug 1, 2023	0 - 0	0	0	0.00	0	0	
496350	Supipara - Chapter 1 Spring Has Come!	Jul 29, 2016	0 - 20000	0	0	5.24	65	0	Spiri Apri the tree:
1034400	Mystery Solitaire The Black Raven	May 6, 2019	0 - 20000	0	0	4.99	0	0	In you th b my
3292190	버튜버 파 라노이아 - Vtuber Paranoia	Oct 31, 2024	0 - 20000	1	0	8.99	0	1	sy 'He Hi You
3631080	Maze Quest VR	Apr 24, 2025	0 - 20000	0	0	4.99	0	0	Its r a M a Er

```

In [6]: # structure of the dataset
# most common value in that column
def top_val(s):
    vc = s.value_counts(dropna=True)
    return str(vc.index[0])[:22] if len(vc) else '(all null)'

# what percentage of all 125855 rows have this value
def top_pct(s):
    vc = s.value_counts(dropna=True)
    return round(vc.iloc[0] / len(s) * 100, 2) if len(vc) else 0.0

overview = pd.DataFrame({
    'dtype'      : df.dtypes,
    'null'       : df.isna().sum(),

```

```

    'pct_filled' : (df.notna().mean() * 100).round(2),
    'n_unique'   : df.nunique(dropna=True),
    'top_value'  : df.apply(top_val),
    'top_pct'    : df.apply(top_pct),
    })

print(overview.sort_values('top_pct', ascending=False).to_string())

```

	dtype	null	pct_filled	n_unique
top_value top_pct				
Windows	bool	0	100.00	2
True 99.97				
User score	int64	0	100.00	31
0 99.97				
Required age	int64	0	100.00	15
0 98.98				
Median playtime two weeks	int64	0	100.00	999
0 96.86				
Average playtime two weeks	int64	0	100.00	995
0 96.86				
Metacritic score	int64	0	100.00	73
0 96.62				
Linux	bool	0	100.00	2
False 87.23				
DLC count	int64	0	100.00	117
0 85.67				
Peak CCU	int64	0	100.00	1110
0 84.36				
Recommendations	int64	0	100.00	5340
0 83.30				
Mac	bool	0	100.00	2
False 82.74				
Median playtime forever	int64	0	100.00	2512
0 79.20				
Average playtime forever	int64	0	100.00	3037
0 79.20				
Discount	int64	0	100.00	88
0 67.57				
Estimated owners	object	0	100.00	14
0 - 20000 60.15				
Full audio languages	object	0	100.00	3785
[] 59.29				
Achievements	int64	0	100.00	449
0 48.57				
Negative	int64	0	100.00	2726
0 46.43				
Supported languages	object	0	100.00	19614
['English'] 45.04				
Positive	int64	0	100.00	5541
0 36.16				

Price		float64	0	100.00	953	
0.0	21.18					
Categories		object	8966	92.88	14058	Single-pl
ayer,Family S	18.97					
Genres		object	8423	93.31	2922	
Casual,Indie	5.49					
Publishers		object	8922	92.91	64607	BFG
Entertainment	0.44					
Support email		object	22630	82.02	62321	info@big
fishgames.com	0.26					
Support url		object	70560	43.94	36161	https://w
ww.facebook.c	0.26					
Tags		object	42502	66.23	77184	Adventur
e,Casual,Hidde	0.22					
Website		object	75335	40.14	40353	https://w
ww.facebook.c	0.22					
Developers		object	8449	93.29	72859	Er
oticGamesClub	0.18					
Notes		object	102545	18.52	19332	This Game
may contain	0.15					
Release date		object	0	100.00	5141	
Oct 23, 2025	0.15					
About the game		object	8462	93.28	116776	Help the
beautiful gir	0.05					
Reviews		object	113674	9.68	11987	“🏆 I lov
ed! ❤️❤️❤️” Pl	0.04					
Screenshots		object	6027	95.21	119718	https://s
hared.akamai.	0.02					
Name		object	1	100.00	124644	Shadow of
the Tomb Rai	0.02					
Score rank		float64	125815	0.03	3	
99.0	0.02					
Metacritic url		object	121597	3.38	4162	https://w
ww.metacritic	0.02					
Header image		object	81	99.94	125664	https://s
hared.akamai.	0.02					
Movies		float64	125855	0.00	0	
(all null)	0.00					

top_pct tell me that if the data worth to add in regardless of how full it is. So from here I kinda know what to drop and what to use to answer the questions of the client.

But hold on I'll come back to this later.

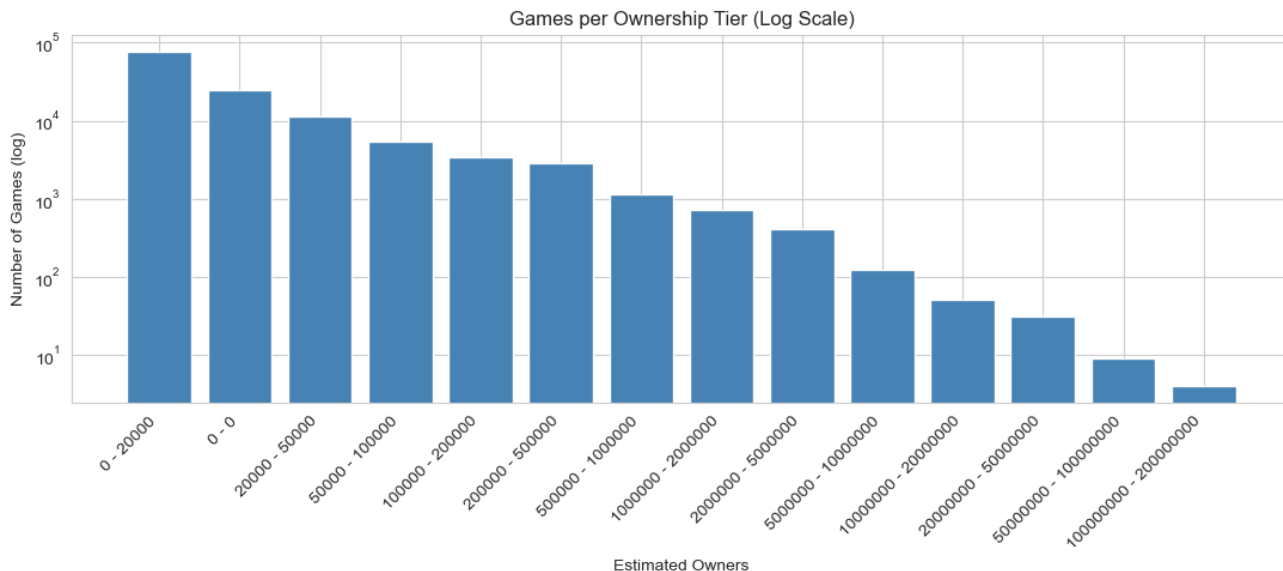
The client stated in the success criteria that "Defines and justifies an ownership-based success threshold based on the observed ownership distribution." and in the dataset limitation the client mentioned "Estimated ownership is reported as a range rather than an exact figure, and most games occupy the lowest ownership bucket". So all of this

shows in the Estimated Owners col. So we'll start from here.

```
In [11]: owners = df['Estimated owners'].value_counts()
key = lambda s: (int(s.split(' - ')[0]), int(s.split(' - ')[1]))
order = sorted(owners.index, key=key)
owners = owners[order]
print(owners)
```

```
Estimated owners
0 - 0                24579
0 - 20000            75705
20000 - 50000        11397
50000 - 100000       5355
100000 - 200000      3454
200000 - 500000      2856
500000 - 1000000     1155
1000000 - 2000000    729
2000000 - 5000000    405
5000000 - 10000000   125
10000000 - 20000000  51
20000000 - 50000000  31
50000000 - 100000000 9
100000000 - 200000000 4
Name: count, dtype: int64
```

```
In [9]: fig, ax = plt.subplots(figsize=(11, 5))
ax.bar(range(len(owners)), owners.values, color='steelblue')
ax.set_yscale('log') # Log scale for y-axis
ax.set_xticks(range(len(owners)))
ax.set_xticklabels(owners.index, rotation=45, ha='right')
ax.set_title('Games per Ownership Tier (Log Scale)')
ax.set_xlabel('Estimated Owners')
ax.set_ylabel('Number of Games (log)')
plt.tight_layout()
plt.show()
```



so what these tell us about. the Estimated owners col sorts every game into one of 14 groups. there are 2 groups that hold almost everything. 0-0 (24579 + 0-20000 (75705) = 100284. it is about ~79.68% of the file. So about 8 out of every 10 games have 20000 owners or even fewer. The biggest single group is 0-200000 with 75705 games like about 60.15% on its own. The figure shows each group above is smaller than the one below. Like after the second group the counts only go down and never up (like a stair). Also, very big games are actually very rare like games with 5000000 owners or more are about 220 games (~0.175%). Literally fewer than 2 games in every 1000.

So from here I would pick the success line at 50000 owners or more (total will be about 14174 games ~11.26%)

So next is to look a bit more closer to 0-0 cause it seems there something hiding here.

```
In [17]: df[df['Estimated owners'] == '0 - 0'].head(10)
```

Out[17]:

	Name	Release date	Estimated owners	Peak CCU	Required age	Price	Discount	DL count
2539430	Black Dragon Mage Playtest	Aug 1, 2023	0 - 0	0	0	0.00	0	
1946890	Codename: Warlock Playtest	Mar 25, 2022	0 - 0	0	0	0.00	0	
2260230	Midnight Dash	Jan 9,	0 - 0	0	0	0.00	0	

2023

2713630	Black Market	Jan 24, 2025	0 - 0	0	0	0.00	0
2349750	CyberVault Playtest	Mar 8, 2023	0 - 0	0	0	0.00	0
2628280	A Night In Omar's Burger Playtest	Oct 4, 2023	0 - 0	0	0	0.00	0
2731180	Blaster Multiplayer	Jan 2, 2024	0 - 0	0	0	0.00	0
3854040	BRUTALISMUS: Dystopia	Jul 31, 2025	0 - 0	0	0	1.39	0
2310500	Ortheo's Interdimensional Jam Session	Aug 10, 2023	0 - 0	0	0	0.00	0
2641000	Spider-Guy: Trapped in the Cheese Place	Feb 21, 2024	0 - 0	0	0	0.00	0

In []:

In []:

In []:

```
In [19]: CORE = ['Genres', 'Categories', 'Developers', 'Publishers', 'About the game']
is_listing_only = df[CORE].isna().all(axis=1)
print("no metadata at all :", is_listing_only.sum())
print("eligible games:", len(df) - is_listing_only.sum())
junk = df[is_listing_only]
print("\nhave Playtest in the name:",
      junk['Name'].str.contains('Playtest', case=False, na=False).sum())
print("\ntheir ownership tiers:")
print(junk['Estimated owners'].value_counts())
```

no metadata at all : 7983
eligible games: 117872

have Playtest in the name: 7858

their ownership tiers:
Estimated owners
0 - 0 7983
Name: count, dtype: int64

now we look for the success rate on the eligible games

```
In [21]: eli = df[~is_listing_only].copy()

owners_low = eli['Estimated owners'].str.split(' - ').str[0].astype(int)
eli['success'] = (owners_low >= 50_000).astype(int)

print("eligible   :", len(eli))
print("successful :", eli['success'].sum())
print("success rate: {:.2f}%".format(eli['success'].mean() * 100))
```

eligible : 117872
successful : 14174
success rate: 12.02%

7983 rows are missing all five information columns. Of these, 7858 (98.4%) have Playtest in the name and all 7983 sit in the 0-0 owners group with no exceptions. A Playtest is a free test version not something anyone can buy so I removed them. That leaves 117872 games. Worth to note that I removed them for missing information not for having zero owners.

```
In [25]: # Genres
# get the avg genres per games
genre_long = (eli[['success']]
              .join(eli['Genres'].fillna('').str.split(',').explode()
                   .str.strip().rename('genre'), how='right'))
genre_long = genre_long[genre_long['genre'] != '']
print("distinct genres:", genre_long['genre'].nunique())
print("avg genres per game:", round(len(genre_long) / eli['Genres'].notna().
```

distinct genres: 33
avg genres per game: 2.88

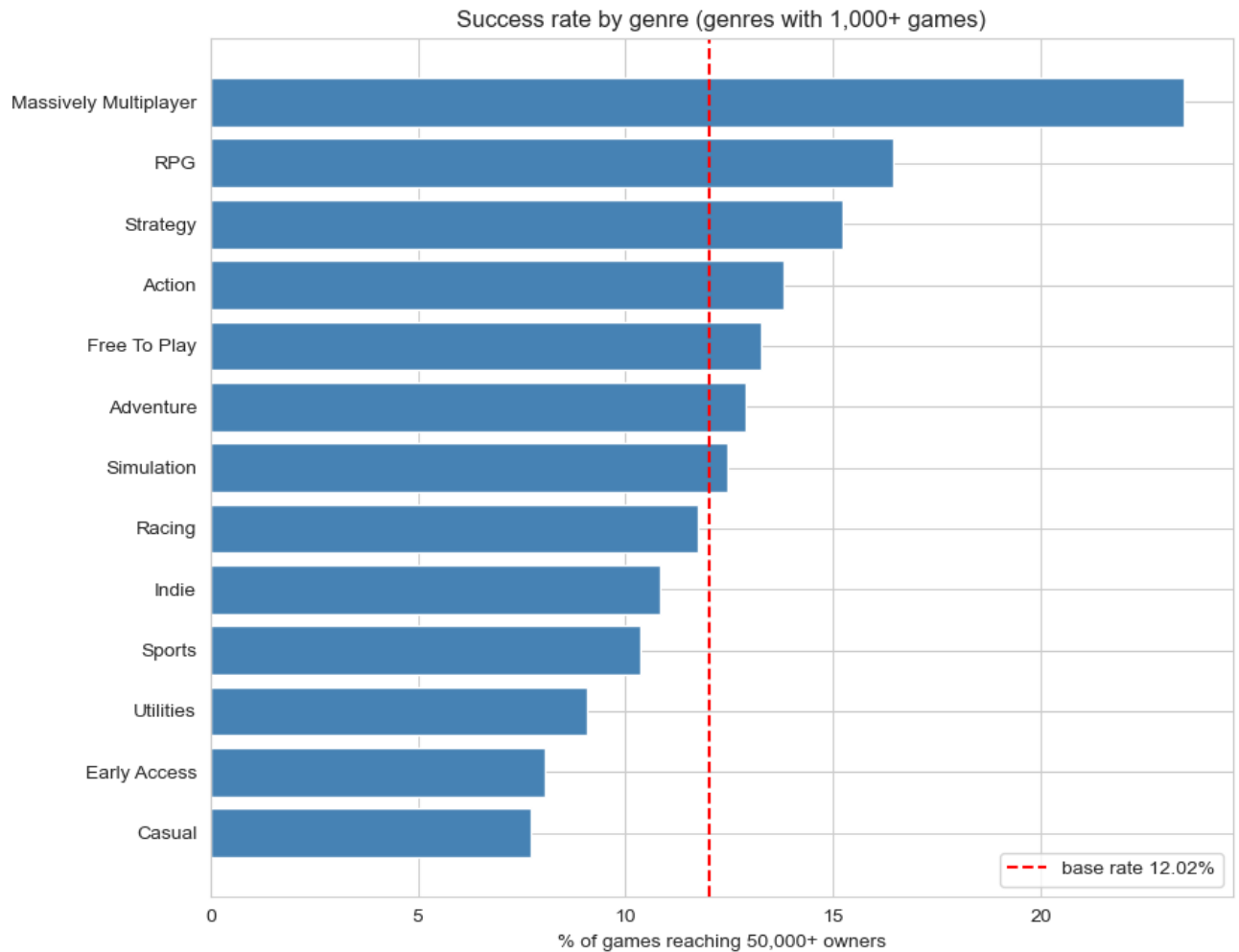
```
In [26]: by_genre =(genre_long.groupby('genre')['success']
          .agg(games='size', wins = 'sum', rate = 'mean'))
by_genre['rate_pct'] = (by_genre['rate'] * 100).round(2)
print("\nbase rate = 12.02%")
print(by_genre.drop(columns = 'rate').sort_values('games', ascending=False).
```

base rate = 12.02%

	games	wins	rate_pct
genre			
Indie	82884	8981	10.84
Casual	51677	3997	7.73
Action	47344	6537	13.81
Adventure	46293	5974	12.90
Simulation	24912	3106	12.47
Strategy	23058	3515	15.24
RPG	21577	3548	16.44
Free To Play	12605	1672	13.26
Early Access	11572	932	8.05
Sports	4971	515	10.36
Racing	4169	490	11.75
Massively Multiplayer	2596	609	23.46
Utilities	1091	99	9.07
Design & Illustration	584	53	9.08
Animation & Modeling	484	42	8.68
Education	464	23	4.96
Violent	414	35	8.45
Video Production	340	31	9.12
Game Development	323	24	7.43
Gore	257	29	11.28
Audio Production	229	17	7.42
Software Training	222	16	7.21
Photo Editing	183	12	6.56
Web Publishing	113	24	21.24
Nudity	94	8	8.51
Sexual Content	88	3	3.41
Accounting	32	5	15.62
Movie	2	1	50.00
Short	1	0	0.00
Tutorial	1	0	0.00
Episodic	1	0	0.00
Documentary	1	0	0.00
360 Video	1	0	0.00

```
In [27]: # visualization
plot = by_genre[by_genre['games'] >= 1000].sort_values('rate_pct')
fig, ax = plt.subplots(figsize=(9, 7))
ax.barh(plot.index, plot['rate_pct'], color='steelblue')
```

```
ax.axvline(12.02, color='red', linestyle='--', label='base rate 12.02%')
ax.set_xlabel('% of games reaching 50,000+ owners')
ax.set_title('Success rate by genre (genres with 1,000+ games)')
ax.legend()
plt.tight_layout()
plt.show()
```

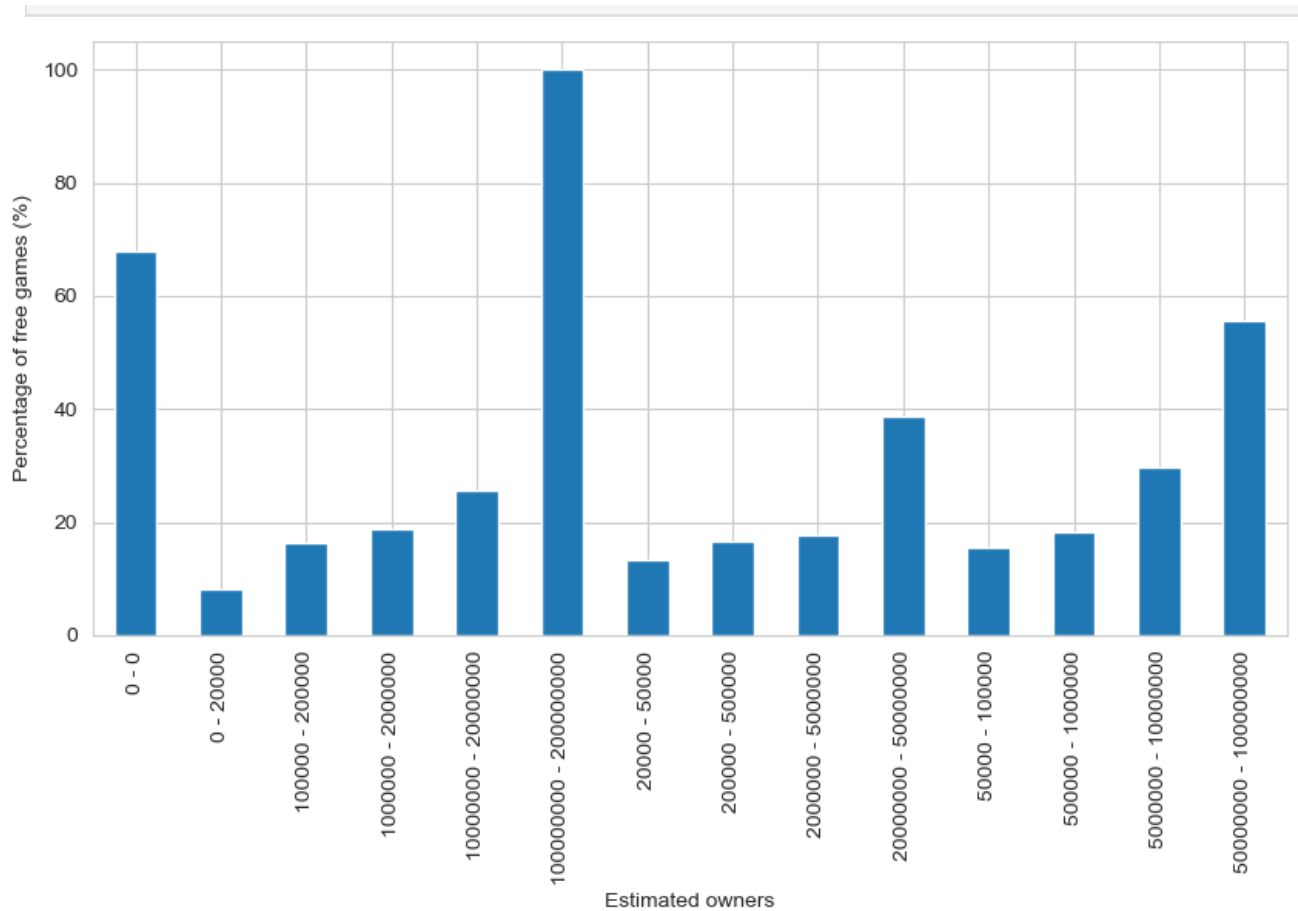


Each game counts once for every genre it has. Only genres with more than 1000 games are shown since small genres can look good by luck alone. What I can see here is that the gap is real as the best genre Massively Multiplayer is about 3 times the worst Casual. This will later help to answer the question of the client that if genre still matters after holding price, release period and platforms steady.

In [28]: *# Price. It can be one of the most things users consider when buying it*
Calculate the fraction of free games (in percent)

```
df_percent_free = df[df['Price'] == 0]['Estimated owners'].value_counts() /
df_wo_free = df[df['Price'] != 0]
```

In [29]: `df_percent_free.plot.bar(ylabel='Percentage of free games (%)');`

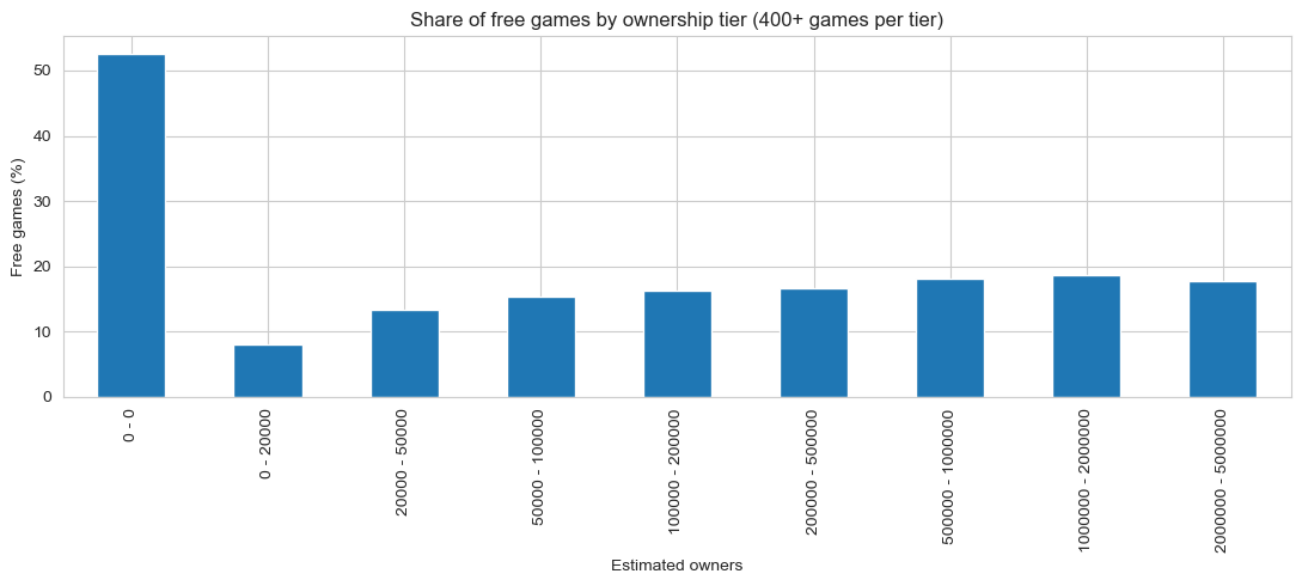


```
In [30]: # some fixes need to be done
# need to sort on both numbers in the range, not on the string
key = lambda s: (int(s.split(' - ')[0]), int(s.split(' - ')[1]))
```

```
In [32]: # all 7983 playtests are free and all sit in 0-0 so inflating from 52.6% to
tot = eli['Estimated owners'].value_counts()
free = eli[eli['Price'] == 0]['Estimated owners'].value_counts()
t = pd.DataFrame({'games': tot, 'free': free}).fillna(0)
t['pct_free'] = (t['free'] / t['games'] * 100).round(1)
t = t.loc[sorted(t.index, key=key)]
print(t.to_string())
```

	games	free	pct_free
Estimated owners			
0 - 0	16596	8724	52.6
0 - 20000	75705	6072	8.0
20000 - 50000	11397	1527	13.4
50000 - 100000	5355	827	15.4
100000 - 200000	3454	563	16.3
200000 - 500000	2856	477	16.7
500000 - 1000000	1155	209	18.1
1000000 - 2000000	729	136	18.7
2000000 - 5000000	405	72	17.8
5000000 - 10000000	125	37	29.6
10000000 - 20000000	51	13	25.5
20000000 - 50000000	31	12	38.7
50000000 - 100000000	9	5	55.6
100000000 - 200000000	4	4	100.0

```
In [34]: # need to drop last 5 groups to have the bars I can trust because with few g
# in the 4 game groups one game is worth 25% point -> cant trust the bar
t_big = t[t['games'] >= 400]
t_big['pct_free'].plot.bar(ylabel='Free games (%)', figsize=(11, 5))
plt.title('Share of free games by ownership tier (400+ games per tier)')
plt.tight_layout()
plt.show()
```



```
In [ ]:
```